

Практическое занятие 2.1

Минимизация квадратичной функции, проклятие размерности и случайный поиск

1. Введение

Цель данного практического занятия — познакомить студентов с фундаментальными проблемами оптимизации многомерных функций, а также научиться применять простейшие алгоритмы случайного поиска для нахождения минимума квадратичных функций. Мы разберём явление «проклятия размерности», исследуем его влияние на вычислительные методы, а затем реализуем несколько вариантов случайного поиска: с фиксированной длиной шага и с убывающей длиной шага.

Выбор квадратичных функций в качестве объекта исследования неслучайный — это один из наиболее важных и простых классов функций в оптимизации. В частности, квадратичные функции часто применяются как тестовые задачи, так как имеют хорошо изученные свойства, гладкость и единственную точку глобального минимума.

Ключевые вопросы занятия:

- Как растёт сложность поиска минимума при увеличении размерности?
- Почему простые методы поиска «наугад» становятся крайне неэффективными в высоких размерностях?
- Как работает случайный поиск и как влияет выбор длины шага?
- Что меняется при переходе к убывающей длине шага?

Во время занятия студенты изучат:

- теоретическую основу задачи минимизации квадратичной функции;
- влияние размерности на плотность точек в гиперкубе;
- роль шага поиска в алгоритмах оптимизации;
- применение случайного поиска для минимизации функции.

Практическая часть включает написание кода, построение графиков и исследование поведения алгоритмов.

2. Теоретические основы

2.1. Квадратичная функция как объект оптимизации

Мы будем исследовать функции вида

$$g(\mathbf{w}) = \mathbf{w}^\top A \mathbf{w},$$

где

- $\mathbf{w} \in \mathbb{R}^N$ — N -мерный вектор,
- A — симметричная положительно определённая матрица, обеспечивающая единственный глобальный минимум в точке $\mathbf{w} = \mathbf{0}$.

Квадратичные функции удобны тем, что обладают:

- аналитически вычисляемым минимумом,
- гладкостью,
- хорошо контролируемой кривизной (через собственные числа матрицы A).

В упражнениях используется простейший случай:

$$g(w_0, w_1) = 100(w_1 - w_0^2)^2 + (1 - w_0)^2,$$

который является двумерным аналогом функций оптимизации из курса машинного обучения.

2.2. Проклятие размерности

Проклятие размерности — это резкое ухудшение поведения алгоритмов при росте размерности входного пространства. Особенно сильно оно проявляется в методах, связанных с перебором, случайным выбором или равномерным покрытием гиперкуба.

Рассмотрим гиперкуб:

$$[-1, 1]^N,$$

который является пространством поиска.

Если мы хотим равномерно покрыть его сеткой с шагом ($h = 0.1$), то число точек равно:

$$K = \left(\frac{2}{h}\right)^N = 20^N.$$

Примеры:

Размерность (N)	Число точек
1	20
2	400
3	8 000

Размерность (N)	Число точек
10	$\sim 10^{13}$
20	$\sim 10^{26}$

Даже при умеренной размерности пространство становится практически бесконечным для перебора.

Это и есть «проклятие»: любая сетка становится слишком разреженной.

2.3. Алгоритмы случайного поиска

Алгоритм случайного поиска имеет вид:

1. Выбрать начальную точку $\mathbf{w}^{(0)}$.
2. На каждом шаге генерировать случайное направление $\mathbf{p}^{(k)}$.
3. Перейти в точку:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \alpha \mathbf{p}^{(k)},$$

где

α — длина шага (фиксированная или убывающая),

$\mathbf{p}^{(k)}$ — случайный вектор в (N)-мерном пространстве, нормированный по длине.

4. Принять новое положение, если оно уменьшает значение функции.

Алгоритм прост, но эффективен только на низких размерностях. В высоких размерностях направления почти никогда не указывают в сторону минимума.

3. Практическая часть

Ниже приведены подробные шаги выполнения упражнений 2.1–2.5.

3.1. Упражнение 2.1 — Исследование проклятия размерности

Цель:

изучить, как растёт число точек, необходимое для равномерного покрытия гиперкуба.

Шаги выполнения

1. Задайте набор размерностей, например:
 $N = 1, 2, 3, 4, 5, 10$.
2. Попробуйте равномерно покрыть гиперкуб $[-1, 1]^N$
с шагом ($h = 0.1$).

3. Используйте формулу:

$$K = \left(\frac{2}{h}\right)^N.$$

4. Постройте график зависимости $K(N)$ в логарифмическом масштабе.

5. Сделайте вывод: почему равномерный перебор почти всегда невозможен?

3.2. Упражнение 2.2 — Минимизация простой квадратичной функции

Задаётся функция:

$$g(\mathbf{w}) = \mathbf{w}^\top A \mathbf{w},$$

где

$$A = \begin{pmatrix} 1 & 0 \\ 0 & 100 \end{pmatrix}.$$

Шаги:

1. Определить функцию $g(w_0, w_1)$.
2. Сгенерировать 100–200 случайных точек в квадрате $[-1, 1]^2$.
3. Вычислить значения функции.
4. Найти минимальное значение.
5. Построить цветовую карту (heatmap).

3.3. Упражнение 2.3 — Реализация случайного поиска

Шаги:

1. Задайте начальную точку:

$$\mathbf{w}^{(0)} = \begin{bmatrix} -1 \\ +1 \end{bmatrix}.$$

2. Определите длину шага:
 $\alpha = 0.1$.

3. На каждом шаге генерируйте случайное направление:

$$\mathbf{p}^{(k)} \sim \mathcal{N}(0, I_N), \quad \mathbf{p}^{(k)} = \frac{\mathbf{p}^{(k)}}{|\mathbf{p}^{(k)}|}.$$

4. Проверяйте условие улучшения:

$$g(\mathbf{w}^{(k+1)}) < g(\mathbf{w}^{(k)}).$$

5. Выполните 1000 шагов.
6. Постройте график уменьшения значения функции.

3.4. Упражнение 2.4 — Сравнение длины шага

Исследуйте влияние разных значений α :

- $\alpha = 1.0$
- $\alpha = 0.1$
- $\alpha = 0.01$

Что сделать:

1. Для каждого α выполните 1000 шагов.
2. Постройте график траектории.
3. Сравните:
 - скорость уменьшения функции,
 - stability поведения,
 - общую производительность.

3.5. Упражнение 2.5 — Убывающая длина шага

Теперь длина шага:

$$\alpha_k = \frac{1}{k}.$$

Шаги:

1. Используйте тот же алгоритм случайного поиска.
2. На (k)-ом шаге уменьшайте длину шага.
3. Постройте график:

$$g(\mathbf{w}^{(k)}), \quad k = 1, \dots, 1000.$$

Выводы:

- Алгоритм становится более стабильным.
- Риск «вылететь» за границы пространства сильно уменьшается.
- Но сходимость замедляется.

4. Исследовательские задания

Каждый студент должен выполнить два задания.

Задание 1

Измените размерность задачи на:

$$N = 5.$$

Проведите эксперимент случайного поиска с:

- фиксированным шагом $\alpha = 0.1$,
- фиксированным шагом $\alpha = 1.0$,
- убывающим шагом $\alpha_k = 1/k$.

Сравните:

- скорость сходимости,
- вариативность результата,
- устойчивость.

Сделайте выводы.

Задание 2

Выберите матрицу:

$$A = \begin{pmatrix} 10 & 0 \\ 0 & 1 \end{pmatrix},$$

вместо исходной.

Исследуйте, как «кривизна» влияет на:

- форму уровня функции,
- поведение случайного поиска,
- чувствительность к длине шага.

5. Контрольные вопросы

Каждый студент должен ответить на два вопроса.

Вопрос 1.

Почему число точек в гиперкубе растёт экспоненциально с размерностью?

Вопрос 2.

Как влияет длина шага (α) на поведение алгоритма случайного поиска?

6. Выводы по практическому занятию

В ходе работы студенты познакомились с фундаментальными проблемами оптимизации многомерных функций, в частности — с проклятием размерности. Было показано, что даже простейший подход, такой как случайный поиск, быстро теряет эффективность при увеличении размерности пространства.

Использование фиксированной длины шага приводит к двум возможным проблемам: слишком большой шаг вызывает «скачки» вокруг минимума, слишком маленький — замедляет сходимость. Введение убывающей длины шага позволяет повысить устойчивость метода, хотя и делает его более медленным.

Студенты получили навыки работы с квадратичными функциями, генерации случайных направлений, построения графиков и анализа поведения алгоритмов оптимизации.

```

{
  "cells": [
    {
      "cell_type": "markdown",
      "metadata": {},
      "source": [
        "\n",
        "# Практическое занятие 2.1 \n",
        "## Минимизация квадратичной функции, «проклятие размерности» и случайный поиск\n",
        "\n",
        "В этом ноутбуке мы шаг за шагом реализуем и исследуем простые алгоритмы минимизации квадратических функций:\n",
        "- иллюстрируем *проклятие размерности* на гиперкубе;\n",
        "- реализуем случайный поиск с фиксированной длиной шага;\n",
        "- реализуем случайный поиск с убывающей длиной шага;\n",
        "- сравниваем поведение методов на простой квадратичной функции и на функции Розенброка.\n",
        "\n",
        "Все пояснения и комментарии, чтобы вы могли использовать ноутбук как интерактивную методичку.\n"
      ]
    },
    {
      "cell_type": "markdown",
      "metadata": {},
      "source": [
        "\n",
        "## План работы\n",
        "\n",
        "1. Импорт библиотек и вспомогательные функции. \n",

```



```

"2. Проклятие размерности: оценка количества точек в гиперкубе при равномерном покрытии.
\n",

"3. Определение тестовых функций: простая квадратичная функция и функция Розенброка. \n",

"4. Реализация алгоритма случайного поиска с фиксированной длиной шага. \n",

"5. Эксперимент: влияние длины шага на сходимость. \n",

"6. Случайный поиск с убывающей длиной шага. \n",

"7. Сравнение результатов и обсуждение. \n",

"8. Индивидуальные исследовательские задания и контрольные вопросы.\n"

]

},

{

"cell_type": "code",

"execution_count": null,

"metadata": {},

"outputs": [],

"source": [

"\n",

"# Импорт необходимых библиотек\n",

"import numpy as np\n",

"import matplotlib.pyplot as plt\n",

"\n",

"# %matplotlib inline # при необходимости раскомментируйте\n",

"\n",

"print(\"Версия numpy:\", np.__version__)\n"

]

},

{

"cell_type": "markdown",

"metadata": {},

```

```

"source": [
    "\n",
    "## 1. Проклятие размерности\n",
    "\n",
    "Рассмотрим гиперкуб  $[-1, 1]^N$  и равномерную сетку с шагом  $h$  по каждой координате.\n",
    "Число точек такой сетки равно\n",
    "\n",
    " $K = (2 / h)^N$ .\n",
    "\n",
    "Даже при умеренном  $N$  значение  $K$  растёт экспоненциально — это и есть одна из форм  

    «проклятия размерности».\n"
]
},
{
    "cell_type": "code",
    "execution_count": null,
    "metadata": {},
    "outputs": [],
    "source": [
        "\n",
        "# шаг сетки\n",
        "h = 0.1\n",
        "Ns = np.array([1, 2, 3, 4, 5, 10])\n",
        "\n",
        " $K = (2.0 / h) ** Ns$ \n",
        "\n",
        "for n, k in zip(Ns, K):\n",
        "    print(f"N = {n:2d}, количество точек K = {k:.3e}")\n"
    ]
}

```

```

},

{
  "cell_type": "code",
  "execution_count": null,
  "metadata": {},
  "outputs": [],
  "source": [
    "\n",
    "# График зависимости K(N) в логарифмическом масштабе\n",
    "\n",
    "plt.figure()\n",
    "plt.plot(Ns, K, marker='o')\n",
    "plt.yscale('log')\n",
    "plt.xlabel('Размерность N')\n",
    "plt.ylabel('Количество точек K (логарифмическая шкала)')\n",
    "plt.title('Экспоненциальный рост количества точек в гиперкубе')\n",
    "plt.grid(True)\n",
    "plt.show()\n"
  ]
},

{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "\n",
    "## 2. Простейшая квадратичная функция\n",
    "\n",
    "Будем работать с простой квадратичной функцией\n",
    "\n",

```

```
"g(w) = ||w||^2 = sum_i w_i^2.\n",
```

```
"\n",
```

```
"Минимум этой функции достигается в точке  $w^* = 0$ , где  $g(w^*) = 0$ .\n"
```

```
]
```

```
},
```

```
{
```

```
"cell_type": "code",
```

```
"execution_count": null,
```

```
"metadata": {},
```

```
"outputs": [],
```

```
"source": [
```

```
"\n",
```

```
"def quadratic(w):\n",
```

```
"    \"\"\"\n",
```

```
"    Простейшая квадратичная функция  $g(w) = ||w||^2$ .\n",
```

```
"    Аргумент:\n",
```

```
"        w : numpy-массив формы (N,) или (1, N)\n",
```

```
"    Возвращает:\n",
```

```
"        скалярное значение  $g(w)$ \n",
```

```
"    \"\"\"\n",
```

```
"    w = np.asarray(w).reshape(-1)\n",
```

```
"    return float(np.dot(w, w))\n",
```

```
"\n",
```

```
"# Примеры\n",
```

```
"examples = [np.array([0.0, 0.0]),\n",
```

```
"            np.array([1.0, 0.0]),\n",
```

```
"            np.array([1.0, 2.0]),\n",
```

```
"            np.array([-1.0, 1.0])]\n",
```

```
"\n",
```

```

"for w in examples:\n",

"    print(f\"w = {w}, g(w) = {quadratic(w):.3f}\")\n"

]

},

{

"cell_type": "markdown",

"metadata": {},

"source": [

"\n",

"## 3. Функция Розенброка (более сложный пример)\n",

"\n",

"В двумерном случае функция Розенброка задаётся формулой\n",

"\n",

" $g(w_0, w_1) = 100 (w_1 - w_0^2)^2 + (1 - w_0)^2$ .\n",

"\n",

"У этой функции узкая изогнутая долина, в которой находится глобальный минимум, поэтому методы оптимизации могут вести себя сложнее, чем на простой квадратичной функции.\n"

]

},

{

"cell_type": "code",

"execution_count": null,

"metadata": {},

"outputs": [],

"source": [

"\n",

"def rosenbrock(w):\n",

"    \"\"\"\n",

"    Функция Розенброка в двумерном случае:\n",

```

```

"    g(w0, w1) = 100 (w1 - w0^2)^2 + (1 - w0)^2\n",
"    \"\"\"\n",
"    w = np.asarray(w).reshape(-1)\n",
"    w0, w1 = w[0], w[1]\n",
"    return float(100.0 * (w1 - w0**2)**2 + (1.0 - w0)**2)\n",
"\n",
"test_points = [np.array([0.0, 0.0]),\n",
"                np.array([1.0, 1.0]),\n",
"                np.array([-1.0, 1.0]),\n",
"                np.array([2.0, 4.0])]\n",
"\n",
"for w in test_points:\n",
"    print(f"w = {w}, g(w) = {rosenbrock(w):.3f}")\n"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"\n",
"## 4. Вспомогательная функция: случайное направление\n",
"\n",
"В алгоритме случайного поиска нам нужно генерировать случайное направление в N-мерном пространстве:\n",
"\n",
"1. генерируем вектор из стандартного нормального распределения;\n",
"2. нормируем его до единичной длины.\n"
]
},

```

```

{
  "cell_type": "code",
  "execution_count": null,
  "metadata": {},
  "outputs": [],
  "source": [
    "\n",
    "def random_direction(dim):\n",
    "    \"\"\" \n",
    "    Генерирует случайное направление в  $R^{dim}$ . \n",
    "    1) берём вектор из  $N(0, 1)$  \n",
    "    2) нормируем его до длины 1 \n",
    "    \"\"\" \n",
    "    v = np.random.randn(dim)\n",
    "    norm = np.linalg.norm(v)\n",
    "    if norm == 0.0:\n",
    "        return random_direction(dim)\n",
    "    return v / norm\n",
    "\n",
    "# Примеры\n",
    "for d in [2, 3]:\n",
    "    print(f"Примеры направлений в  $R^{{d}}$ :")\n",
    "    for _ in range(3):\n",
    "        print(random_direction(d))\n",
    "    print()\n",
  ]
},
{
  "cell_type": "markdown",

```

```

"metadata": {},

"source": [

    "\n",

    "## 5. Случайный поиск с фиксированной длиной шага\n",

    "\n",

    "Алгоритм случайного поиска для минимизации функции g(w):\n",

    "\n",

    "1. Задать начальную точку w^0.\n",

    "2. Для k = 0, 1, ..., K-1:\n",

    "    - сгенерировать случайное направление p^k;\n",

    "    - сформировать кандидата w_tilde = w^k + alpha * p^k;\n",

    "    - если g(w_tilde) < g(w^k), принять шаг (w^{k+1} = w_tilde), иначе оставить w^{k+1} = w^k.\n",

    ]

},

{

    "cell_type": "code",

    "execution_count": null,

    "metadata": {},

    "outputs": [],

    "source": [

        "\n",

        "def random_search(g, w0, alpha=0.1, max_its=1000, print_every=0):\n",

        "    \"\"\"\n",

        "        Случайный поиск с фиксированной длиной шага.\n",

        "    \"\"\"\n",

        "    w = np.asarray(w0).astype(float).reshape(-1)\n",

        "    dim = w.size\n",

        "\n",

        "    w_hist = []\n",

```



```

"    g_hist = []\n",

"\n",

"    for k in range(max_its):\n",

"        w_hist.append(w.copy())\n",

"        g_hist.append(g(w))\n",

"\n",

"        p = random_direction(dim)\n",

"        w_candidate = w + alpha * p\n",

"        g_candidate = g(w_candidate)\n",

"        g_current = g_hist[-1]\n",

"\n",

"        if g_candidate < g_current:\n",

"            w = w_candidate\n",

"\n",

"        if print_every and (k % print_every == 0):\n",

"            print(f"iter {k:4d}: g(w) = {g(w):.6f}")\n",

"\n",

"        w_hist.append(w.copy())\n",

"        g_hist.append(g(w))\n",

"\n",

"    return np.array(w_hist), np.array(g_hist)\n"

]

},

{

"cell_type": "markdown",

"metadata": {},

"source": [

"\n",

"### 5.1. Эксперимент на квадратичной функции\n",

```

```

"\n",

"Применим случайный поиск к функции  $g(w) = ||w||^2$  из точки  $w_0 = [-1, 1]$  с шагом  $\alpha = 0.1$ .\n"

]

},

{

"cell_type": "code",

"execution_count": null,

"metadata": {},

"outputs": [],

"source": [

"\n",

"w0 = np.array([-1.0, 1.0])\n",

"alpha = 0.1\n",

"max_its = 200\n",

"\n",

"w_hist_quad, g_hist_quad = random_search(quadratic, w0, alpha=alpha, max_its=max_its)\n",

"\n",

"print(\"Начальная точка:\", w0)\n",

"print(\"Начальное значение  $g(w_0)$  =\", quadratic(w0))\n",

"print(\"Конечная точка:\", w_hist_quad[-1])\n",

"print(\"Конечное значение  $g(w_T)$  =\", g_hist_quad[-1])\n",

"\n",

"plt.figure()\n",

"plt.plot(g_hist_quad)\n",

"plt.xlabel(\"Номер итерации k\")\n",

"plt.ylabel(\" $g(w^k)$ \")\n",

"plt.title(\"Случайный поиск на квадратичной функции (фиксированный шаг)\")\n",

"plt.grid(True)\n",

```

```

"plt.show()\n"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"\n",
"### 5.2. Влияние длины шага alpha\n",
"\n",
"Сравним поведение алгоритма для нескольких значений шага:\n",
"- alpha = 1.0\n",
"- alpha = 0.1\n",
"- alpha = 0.01\n",
]
},
{
"cell_type": "code",
"execution_count": null,
"metadata": {},
"outputs": [],
"source": [
"\n",
"alphas = [1.0, 0.1, 0.01]\n",
"max_its = 300\n",
"\n",
"plt.figure()\n",
"for alpha in alphas:\n",
"    _, g_hist = random_search(quadratic, w0, alpha=alpha, max_its=max_its)\n",

```

```

plt.plot(g_hist, label=f"alpha = {alpha}")\n",
"\n",
plt.xlabel("Номер итерации k")\n",
plt.ylabel("g(w^k)")\n",
plt.title("Влияние длины шага на сходимость случайного поиска (квадратичная функция)")\n",
plt.legend()\n",
plt.grid(True)\n",
plt.show()\n"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"\n",
"## 6. Случайный поиск с убывающей длиной шага\n",
"\n",
"Теперь будем уменьшать длину шага по мере роста номера итерации k, например по
правилу\n",
"\n",
"alpha_k = alpha0 / (k + 1).\n"
]
},
{
"cell_type": "code",
"execution_count": null,
"metadata": {},
"outputs": [],
"source": [

```

```

"\n",

"def random_search_diminishing(g, w0, alpha0=1.0, max_its=1000, print_every=0):\n",

"    \"\"\"\n",

"    Случайный поиск с убывающей длиной шага:  $\alpha_k = \alpha_0 / (k + 1)$ .\n",

"    \"\"\"\n",

"    w = np.asarray(w0).astype(float).reshape(-1)\n",

"    dim = w.size\n",

"\n",

"    w_hist = []\n",

"    g_hist = []\n",

"\n",

"    for k in range(max_its):\n",

"        w_hist.append(w.copy())\n",

"        g_hist.append(g(w))\n",

"\n",

"        alpha_k = alpha0 / (k + 1)\n",

"        p = random_direction(dim)\n",

"        w_candidate = w + alpha_k * p\n",

"        g_candidate = g(w_candidate)\n",

"        g_current = g_hist[-1]\n",

"\n",

"        if g_candidate < g_current:\n",

"            w = w_candidate\n",

"\n",

"        if print_every and (k % print_every == 0):\n",

"            print(f\"iter {k:4d}: g(w) = {g(w):.6f}, alpha_k = {alpha_k:.4f}\")\n",

"\n",

"        w_hist.append(w.copy())\n",

"        g_hist.append(g(w))\n",

```

```

"\n",
"    return np.array(w_hist), np.array(g_hist)\n"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"\n",
"### 6.1. Сравнение фиксированного и убывающего шага на функции Розенброка\n"
]
},
{
"cell_type": "code",
"execution_count": null,
"metadata": {},
"outputs": [],
"source": [
"\n",
"w0_rb = np.array([-1.0, 1.0])\n",
"max_its = 1000\n",
"\n",
"_, g_hist_fixed = random_search(rosenbrock, w0_rb, alpha=0.01, max_its=max_its)\n",
"_, g_hist_dim = random_search_diminishing(rosenbrock, w0_rb, alpha0=0.1, max_its=max_its)\n",
"\n",
"plt.figure()\n",
"plt.plot(g_hist_fixed, label=\"фиксированный шаг (alpha = 0.01)\")\n",
"plt.plot(g_hist_dim, label=\"убывающий шаг (alpha0 = 0.1)\")\n",
"plt.yscale(\"log\")\n",

```

```

plt.xlabel("\u041d\u043e\u043c\u0435\u0440 \u0438\u0442\u0435\u0440\u0430\u0446\u0438\u0438 k")\n",

plt.ylabel("\u0433(w^k) (логарифмическая шкала)")\n",

plt.title("\u0421\u043b\u0443\u0447\u0430\u0439\u043d\u044b\u0439 \u043f\u043e\u0438\u0441\u043a \u043d\u0430 \u0444\u0443\u043d\u043a\u0446\u0438\u0438 \u0420\u043e\u0437\u0435\u043d\u0431\u0440\u043e\u043a")\n",

plt.legend()\n",

plt.grid(True)\n",

plt.show()\n"

]

},

{

"cell_type": "markdown",

"metadata": {},

"source": [

"\n",

"## 7. Индивидуальные исследовательские задания\n",

"\n",

"Каждый студент выбирает **две** задачи (например, по номеру в списке группы) и выполняет их в этом ноутбуке.\n",

"\n",

"1. Измените размерность задачи:  $N = 2, 5, 10$ . Повторите эксперименты случайного поиска для квадратичной функции из разных начальных точек. Сравните скорость уменьшения значения функции и сделайте вывод о влиянии размерности.\n",

"\n",

"2. Вместо правила  $\alpha_k = \alpha_0 / (k + 1)$  попробуйте другие законы убывания шага, например  $\alpha_k = \alpha_0 / \sqrt{k + 1}$  или  $\alpha_k = \alpha_0 / (k + 1)^\gamma$ , где  $0 < \gamma < 1$ . Сравните графики сходимости и обсудите, какой закон убывания кажется более удачным.\n",

"\n",

"3. Добавьте в алгоритм ограничение области поиска: после каждого шага проецируйте точку в квадрат  $[-2, 2]^2$  (или гиперкуб  $[-2, 2]^N$ ). Сравните поведение алгоритма с вариантом без ограничений.\n",

"\n",

```

"4. Для функции Розенброка выберите несколько начальных точек (например, [-2, 2], [0, 0], [2, 2]) и сравните, как меняется траектория и конечное значение функции при одинаковых параметрах случайного поиска.\n",

"\n",

"5. Исследуйте зависимость качества найденного решения от числа итераций max_its. Постройте график (или таблицу) зависимости среднего значения $g(w)$ от max_its (например, усредняя по 10 запускам). Обсудите, когда увеличение числа итераций перестаёт существенно улучшать результат.\n"

]

},

{

"cell_type": "markdown",

"metadata": {},

"source": [

"\n",

"## 8. Контрольные вопросы\n",

"\n",

"1. Почему равномерное покрытие гиперкуба при фиксированном шаге приводит к экспоненциальному росту числа точек с ростом размерности? Приведите формулу для числа точек и поясните, как она связана с «проклятием размерности».\n",

"\n",

"2. Как влияет длина шага α на поведение алгоритма случайного поиска? Что происходит при слишком большом и при слишком маленьком шаге?\n",

"\n",

"3. Какова интуитивная идея использования убывающей длины шага в случайном поиске? Почему это может улучшить стабильность алгоритма?\n",

"\n",

"4. Чем квадратичная функция $\|w\|^2$ отличается от функции Розенброка с точки зрения сложности оптимизации? Почему на функции Розенброка алгоритмы нередко ведут себя сложнее?\n"

]

},

{


```
"cell_type": "markdown",

"metadata": {},

"source": [

    "\n",

    "## 9. Итоги\n",

    "\n",

    "В этом практическом занятии вы:\n",

    "- увидели проявление проклятия размерности на примере равномерной сетки в гиперкубе;\n",

    "- реализовали и исследовали алгоритм случайного поиска с фиксированным шагом;\n",

    "- реализовали вариант с убывающей длиной шага и сравнили его с фиксированным;\n",

    "- провели эксперименты на простой квадратичной функции и на функции Розенброка;\n",

    "- получили набор исследовательских задач и контрольных вопросов для закрепления материала.\n",

    "\n",

    "Сохраните этот ноутбук (например, в формате PDF) вместе с выполненными заданиями и ответами на вопросы — это и будет вашим отчётом по практическому занятию 2.1.\n"

],

"metadata": {

    "kernelspec": {

        "display_name": "Python 3 (ipykernel)",

        "language": "python",

        "name": "python3"

    },

    "language_info": {

        "codemirror_mode": {

            "name": "ipython",

            "version": 3
```

```
,
"file_extension": ".py",
"mimetype": "text/x-python",
"name": "python",
"nbconvert_exporter": "python",
"pygments_lexer": "ipython3",
"version": "3.8.10"
},
"toc": {
  "colors": {
    "hover_highlight": "#DAA520",
    "navigate_num": "#000000",
    "navigate_text": "#333333",
    "running_highlight": "#FF0000",
    "selected_highlight": "#FFD700",
    "sidebar_border": "#EEEEEE",
    "wrapper_background": "#FFFFFF"
  },
  "moveMenuLeft": true,
  "nav_menu": {
    "height": "216.233px",
    "width": "252px"
  },
  "navigate_menu": true,
  "number_sections": false,
  "sideBar": true,
  "threshold": 4,
  "toc_cell": true,
  "toc_position": {
```

```
"height": "631.733px",  
  
"left": "0px",  
  
"right": "1228px",  
  
"top": "116.267px",  
  
"width": "212px"  
  
},  
  
"toc_section_display": "block",  
  
"toc_window_display": false,  
  
"widenNotebook": false  
  
}  
  
},  
  
"nbformat": 4,  
  
"nbformat_minor": 2  
  
}
```